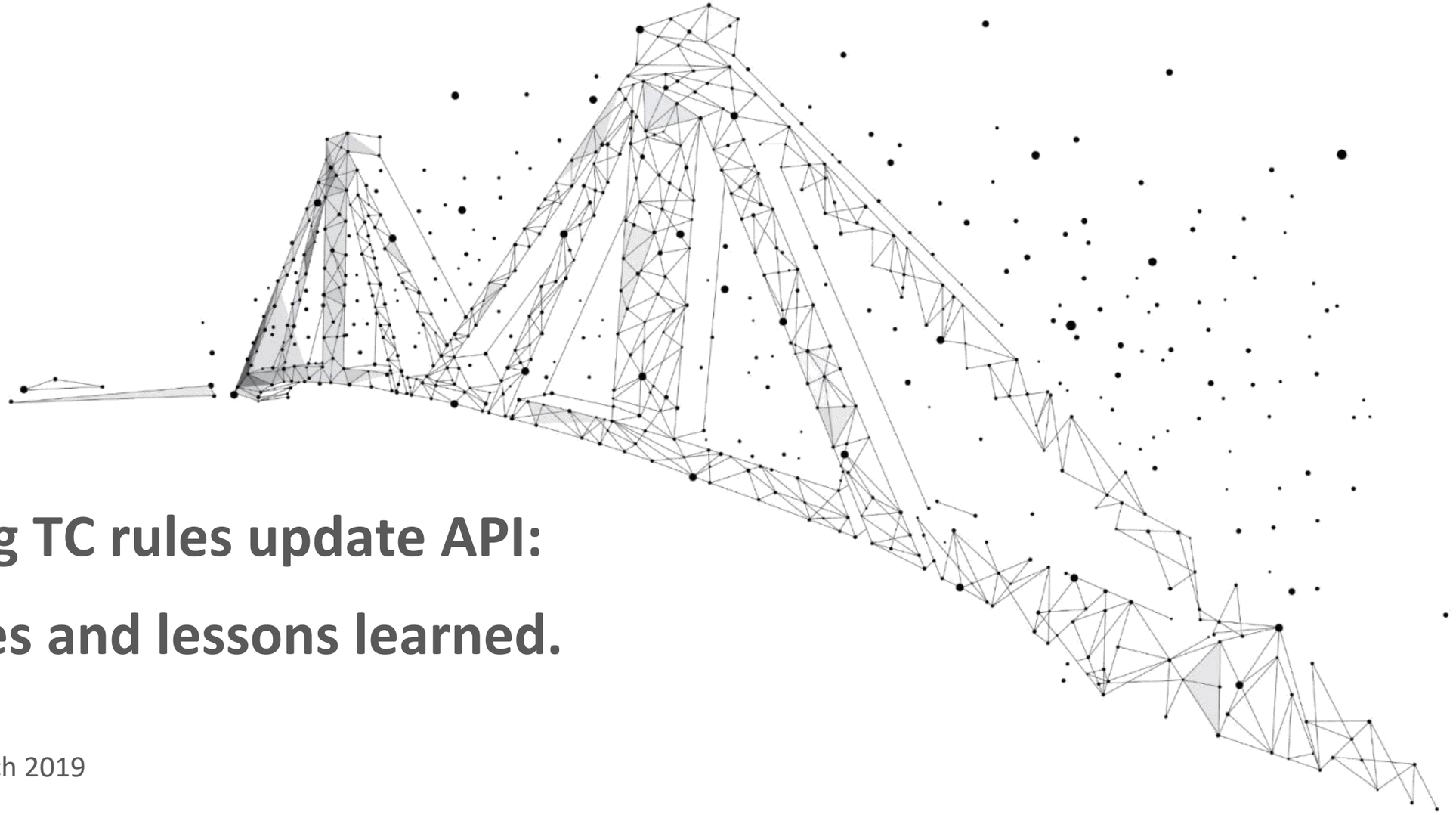


Unlocking TC rules update API: challenges and lessons learned.

Vlad Buslov

NetDev 0x13 March 2019

© 2019 Mellanox Technologies

Recap

Unlocking TC rule update path

© 2019 Mellanox Technologies

Short recap of proposed changes

Before

- TC kernel infrastructure is synchronized by One Big Lock(rtnl lock) that protects all netlink subsystem.
- Seemingly unrelated task like changing IP address of interface and creating/deleting TC rules exclude each other.
- Rtnl lock can be held for seconds by some heavy operations.
- As a result, rule update path doesn’t support any kind of parallelism.

After

- TC rules can be updated in parallel.
- All layers that participate in rule TC rule update are extended with necessary primitives to support unlocked execution:
  - Reference counting
  - Fine-grained locking
  - RCU
- Call driver offload callbacks in parallel.

© 2019 Mellanox Technologies

Upstreaming status

- Actions API change – upstreamed
- Action implementations refactoring – upstreamed
- Classifier API part 1 – upstreamed
- Classifier API part 2 – upstreamed
- Unlocked flower – under review
- Hardware offloads API - pending

© 2019 Mellanox Technologies

Challenges and lessons learned

© 2019 Mellanox Technologies

Asynchronous vs synchronous approach

- Asynchronous:
  - Fast.
  - Can use spinlocks and defer sleeping operations to asynchronous context (workqueue).
  - Hard to reason about (KASAN use-after-free incoming)
- Synchronous:
  - Slower.
  - Needs “heavy” locks that allow sleeping. (discussed in next slide)
  - Easy to reason about.
- Don’t async unless absolutely necessary.
- Don’t be too fancy or clever with locking in general, unless you have benchmark results to justify it.

© 2019 Mellanox Technologies

Spinlock vs Mutex

- Is mutex really slower than spinlock in common case?
- Maintainer requested to benchmark PoC mutex-based version of proposed change that used spinlock. ([PATCH net-next v2 01/17] net: sched: refactor mini_qdisc_pair_swap() to use workqueue)
- Roughly the same performance:
  - 1-3% difference for uncongested case.
  - Up to 17% in highly congested case.
- Use mutex by default, unless spinlock is explicitly required by design. (need to obtain it in atomic context, etc.)

© 2019 Mellanox Technologies

Mutex “optimistic spinning” optimization

+	91.83%	0.00%	tc	[kernel.vmlinux]	[k] entry_SYSCALL_64_after_hwframe
+	91.83%	0.00%	tc	[kernel.vmlinux]	[k] do_syscall_64
+	91.80%	0.00%	tc	libc-2.25.so	[.] __libc_sendmsg
+	91.78%	0.00%	tc	[kernel.vmlinux]	[k] __sys_sendmsg
+	91.77%	0.00%	tc	[kernel.vmlinux]	[k] ___sys_sendmsg
+	91.77%	0.00%	tc	[kernel.vmlinux]	[k] sock_sendmsg
+	91.77%	0.00%	tc	[kernel.vmlinux]	[k] netlink_sendmsg
+	91.77%	0.01%	tc	[kernel.vmlinux]	[k] netlink_unicast
+	91.77%	0.00%	tc	[kernel.vmlinux]	[k] netlink_rcv_skb
+	91.71%	0.01%	tc	[kernel.vmlinux]	[k] rtnetlink_rcv_msg
+	91.69%	0.03%	tc	[kernel.vmlinux]	[k] tc_new_tfilter
+	90.96%	26.45%	tc	[kernel.vmlinux]	[k] __tcf_chain_get
+	64.30%	0.01%	tc	[kernel.vmlinux]	[k] __mutex_lock.isra.7
+	39.36%	39.18%	tc	[kernel.vmlinux]	[k] osq_lock
+	24.92%	24.81%	tc	[kernel.vmlinux]	[k] mutex_spin_on_owner

© 2019 Mellanox Technologies

Implement tests for your changes

- Make sure your changes pass existing tests first!
- Maintainers trust risky changes more, if they are submitted with tests.
- Tc-testing provides mature infrastructure for implementing tests. (Python-based, but you probably will not need to write a single line of Python code for your test)
- Run all tests (selftests tc-testing and forwarding) with KASAN and lock debugging before submitting big changes.

© 2019 Mellanox Technologies

Write descriptive cover letters

- Cover letter is very important for large patch sets.
- People might not review the code if they don’t understand what the code supposed to do after reading cover letter.
- It is discouraging when your cover letter is discussed instead of your code.
- Re-read your cover letter from PoV of person who didn’t read the code yet.

© 2019 Mellanox Technologies

Submit early

- Don’t count on your 17 patch set to be accepted last Friday during rc7 – you will most likely have to resubmit it after merge window.
- Big patch sets don’t get reviewed instantly. (1-2 weeks for feedback)
- Another major change might be accepted earlier and require non trivial merge, sometimes redesign.
- More time to fix the bugs while change is still net-next.

© 2019 Mellanox Technologies

General approach to upstreaming large changes

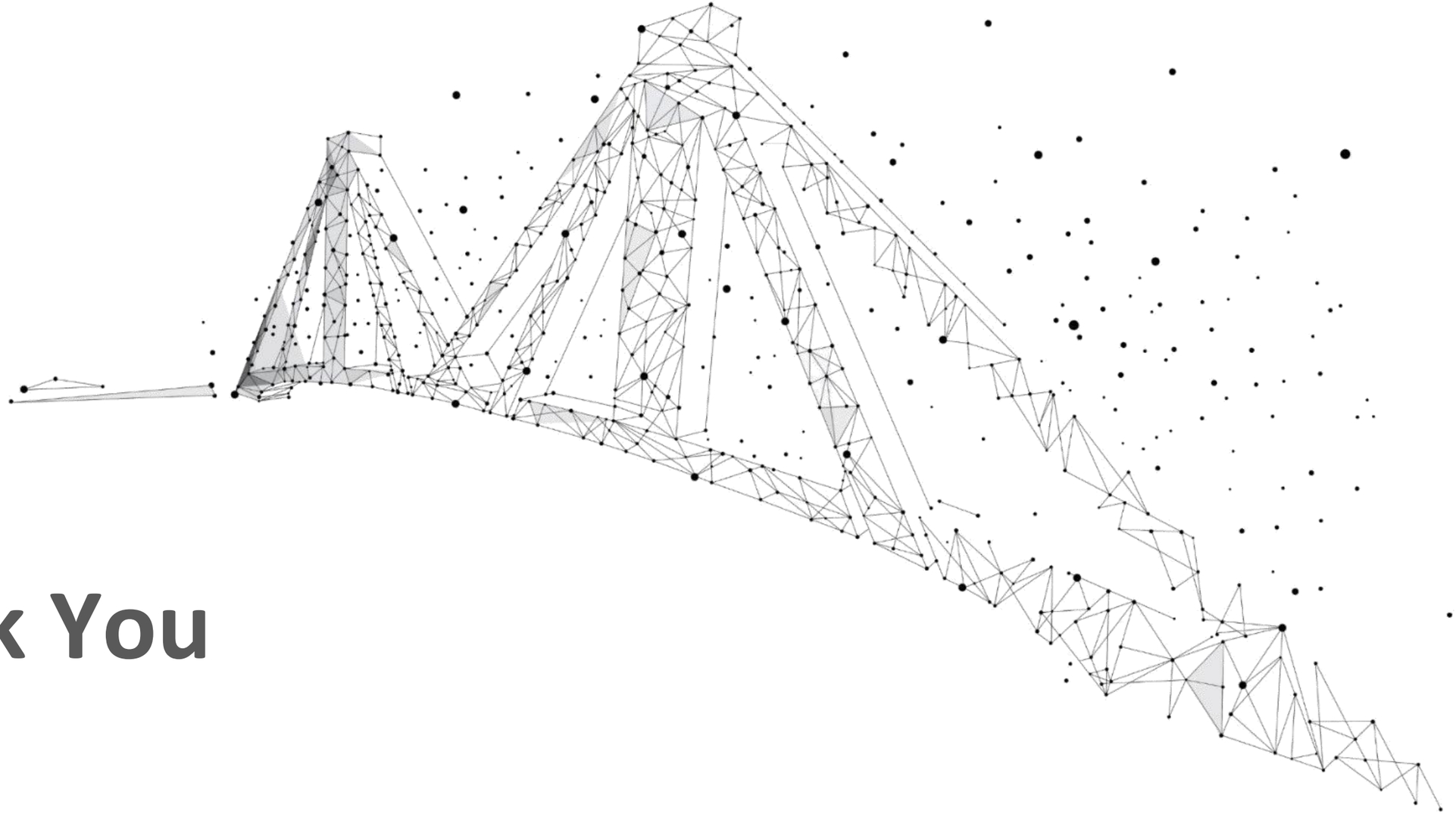
Whole solution

- Can’t be submitted as single set of patches, so you have to submit multiple sets one by one anyway.
- Has an upkeep cost by forcing author to periodically rebase on upstream, sometimes performing major merges/redesign.
- Allows to find problems that are not obvious by just looking at the design.
- Allows to test whole solution.

Start with a design and implement necessary changes one by one

- Easy to get wrong when upstreamed patch set doesn’t implement what you actually need to implement the next one.
- Can’t test the whole change end to end.
- No need to chase and merge with conflicting upstream changes.

© 2019 Mellanox Technologies



Thank You